



Credit Card Fraud Detection EDA

Exploratory Data Analysis & Machine
Learning

Presented by Amaan Uddin

Project Overview

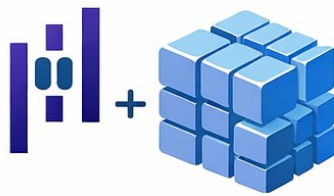
- Analysis of 136,755 real-world credit card transactions
- Objective is to identify and analyze fraudulent behavior
- Dataset contains transaction, merchant, and customer details
- Binary target variable: is_fraud (0 / 1)
- Fraud detection is a high-risk business problem
- Exploratory Data Analysis used for insights
- Focus on amount, category, and merchant features
- Machine learning used for prediction
- Realistic and highly imbalanced dataset
- Goal is data-driven fraud prevention

Tools & Technologies Used

- Python for analysis and modeling
- Pandas and NumPy for data manipulation
- Matplotlib and Seaborn for visualization
- Scikit-learn for machine learning models
- Jupyter Notebook for experimentation



Python



Pandas & NumPy



Matplotlib



Seaborn



Scikit-learn



Jupyter

Dataset Overview

- Total transactions: 1296675
- Fraud transactions: 7,506
- Non-fraud transactions: 1289169
- Fraud rate: 5.78%
- Dataset is highly imbalanced

```
df["is_fraud"].value_counts()
```

```
✓ 0.0s
```

```
is_fraud
```

```
0    1289169
```

```
1       7506
```

```
Name: count, dtype: int64
```

```
df.shape
```

```
(1296675, 23)
```

Data Quality Validation

- Performed column-wise null value analysis to assess dataset completeness
- Confirmed absence of missing values across all transaction attributes
- Checked for duplicate transaction records to ensure data integrity
- Verified that no duplicate rows were present in the dataset

```
df.isnull().sum()
```

```
Unnamed: 0      0
trans_date_trans_time  0
cc_num          0
merchant        0
category       0
amt            0
first          0
last           0
gender         0
street         0
city           0
state          0
zip            0
lat            0
long           0
city_pop       0
job            0
dob            0
trans_num      0
unix_time      0
merch_lat      0
merch_long     0
is_fraud       0
dtype: int64
```

```
df.duplicated().sum()
```

```
np.int64(0)
```

Data Preprocessing & Transformation

- Performed column-wise null value scan
- No critical missing values found
- Dropped non-informative columns
- Avoided unnecessary imputation
- Final dataset ready for EDA

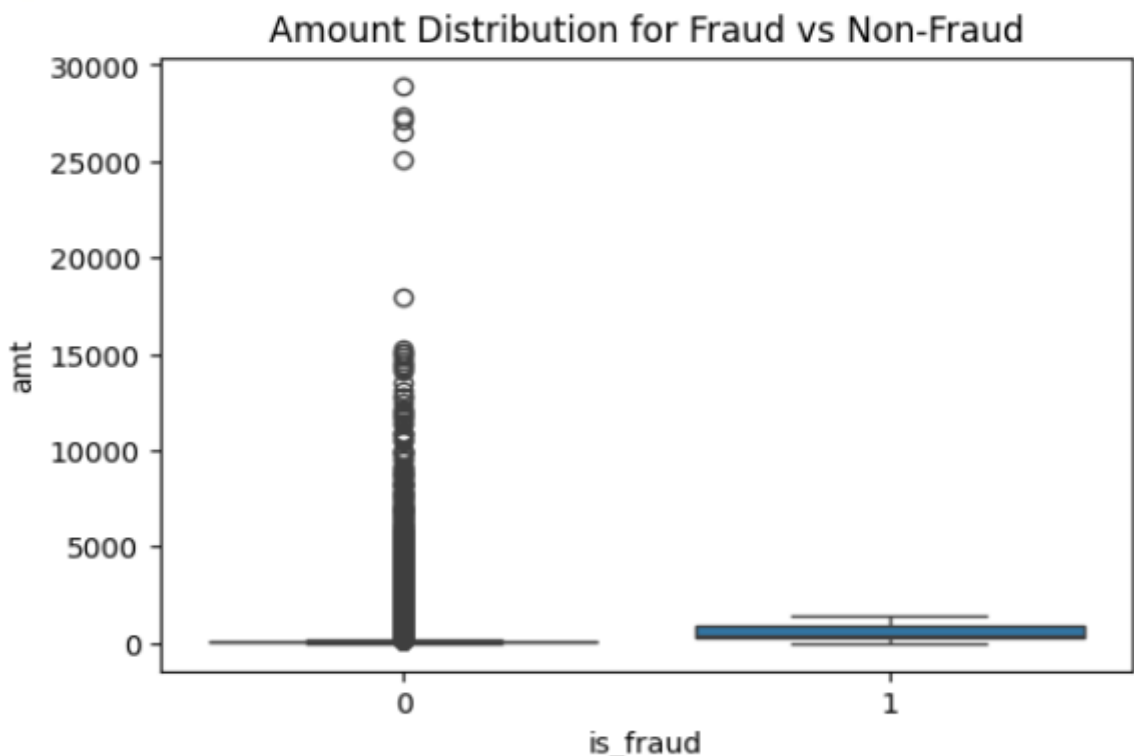
```
df.drop_duplicates(inplace= True)
```

```
drop_cols = ["Unnamed: 0", "first", "last",  
            "street", "job", "dob",  
            'cc_num', 'trans_num']  
df = df.drop(columns=drop_cols, errors='ignore')
```

```
df["trans_date_trans_time"] = pd.to_datetime(  
    df["trans_date_trans_time"])
```

Transaction Amount Distribution

- Non-fraud mean transaction amount: **\$67.67**
- Non-fraud median transaction amount: **\$47.28**
- Fraud mean transaction amount: **\$531.32**
- Fraud median transaction amount: **\$396.50**
- Fraud transactions are approximately **8× higher**



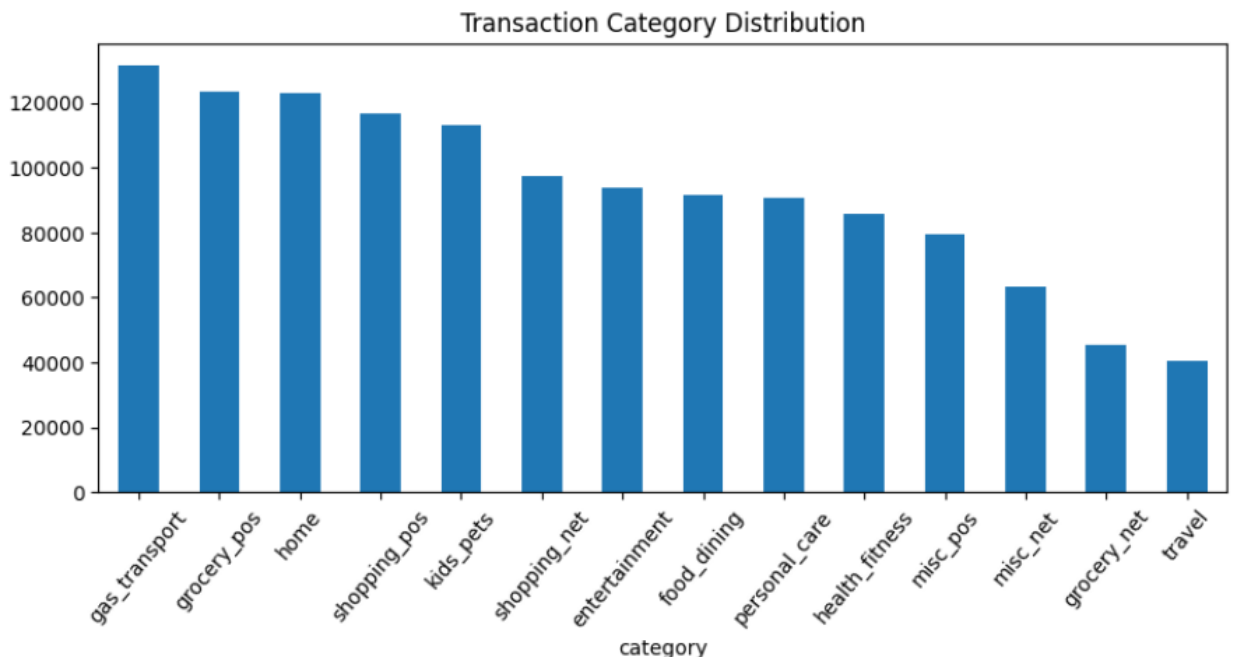
Fraud vs Non-Fraud Distribution

- Non-fraud transactions: 94.21%
- Fraud transactions: 5.79%
- Class imbalance ratio is approximately 17:1
- Accuracy alone is misleading
- Recall and F1-score are more important



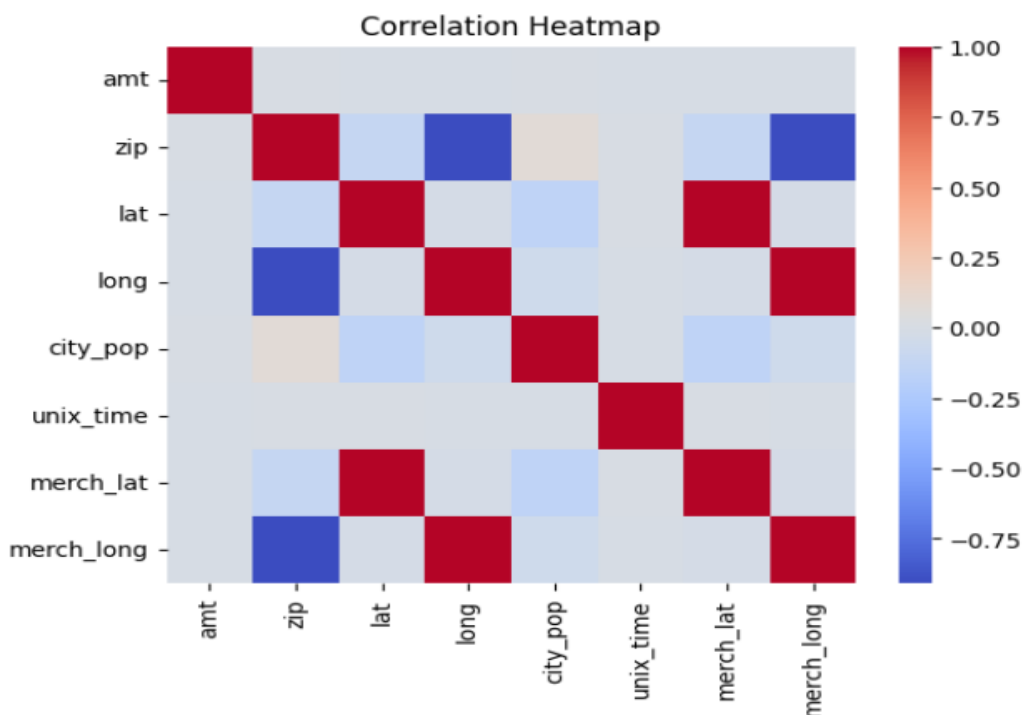
Category-wise Fraud Analysis

- **Gas transport, grocery POS, and home** categories have the highest transaction volumes
- Online and POS categories dominate overall transaction activity
- High transaction volume categories require closer fraud monitoring
- Transaction distribution varies significantly across categories
- High transaction volume does not directly imply high fraud risk



Correlation Heatmap

- Most numerical features show weak linear correlation with each other
- Strong correlation exists between latitude and merchant latitude
- Strong correlation exists between longitude and merchant longitude
- City population and transaction time show minimal correlation
- Low linear correlation supports use of non-linear ML models



Key Fraud Patterns

- Fraud transactions have a significantly higher average amount (~\$531) compared to non-fraud transactions (~\$68)
- Overall fraud rate in the dataset is ~5.78%, indicating strong class imbalance
- A small number of categories contribute to the majority of fraud cases
- Grocery POS (1,743 cases) and Shopping Net (1,713 cases) are the most fraud-prone categories
- Fraud transactions are distributed across multiple merchants rather than a single source
- Transaction amount (amt) is the strongest numerical indicator of fraudulent activity
- Category information provides additional contextual risk beyond transaction value
- Severe class imbalance makes accuracy an unreliable evaluation metric

Machine Learning Approach

- Fraud detection framed as a binary classification problem
- Categorical features encoded using label encoding
- Numerical features scaled using standardization
- Train–test split applied with stratified sampling
- Class imbalance handled using SMOTE resampling
- Multiple models trained on the same feature set
- Model performance evaluated using recall and F1-score

Feature Engineering & Data Preparation

- Categorical variables encoded using label encoding
- Numerical features standardized for model stability
- Feature matrix constructed by combining encoded and scaled features
- Target variable defined as fraud label

```
# Encoding the Categorical Columns
le = LabelEncoder()
cat_df= df.select_dtypes(include= 'object')

for col in cat_df.columns:
    cat_df[col] = le.fit_transform(df[col])

# Feature Scalling on Numeric Columns
scl = StandardScaler()
a= df.select_dtypes(include= ['int', 'float'])
a= a.drop('is_fraud', axis=1 )
num_df = scl.fit_transform(a)
num_df = pd.DataFrame(num_df, columns= [
    'amt', 'zip', 'lat', 'long', 'city_pop', 'unix_time', 'merch_lat', 'merch_long'])

X= pd.concat([num_df, cat_df], axis=1)
y = df["is_fraud"]
```

Train-Test Split & Class Imbalance Handling

- Stratified train–test split applied to preserve class distribution
- Severe class imbalance identified in training data
- SMOTE used to synthetically balance fraud and non-fraud classes
- Balanced dataset prepared for fair model training

```
# Test and Train Split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.17, random_state=32, stratify=y)

# Applying smote to handle imbalanced data

from imblearn.over_sampling import SMOTE

sm = SMOTE(random_state=23)
X_train_res, y_train_res = sm.fit_resample(X_train, y_train)

print("Before SMOTE:", y_train.value_counts())
print("After SMOTE:", y_train_res.value_counts())
```

```
Before SMOTE: is_fraud
0    1070010
1      6230
Name: count, dtype: int64
After SMOTE: is_fraud
0    1070010
1    1070010
```

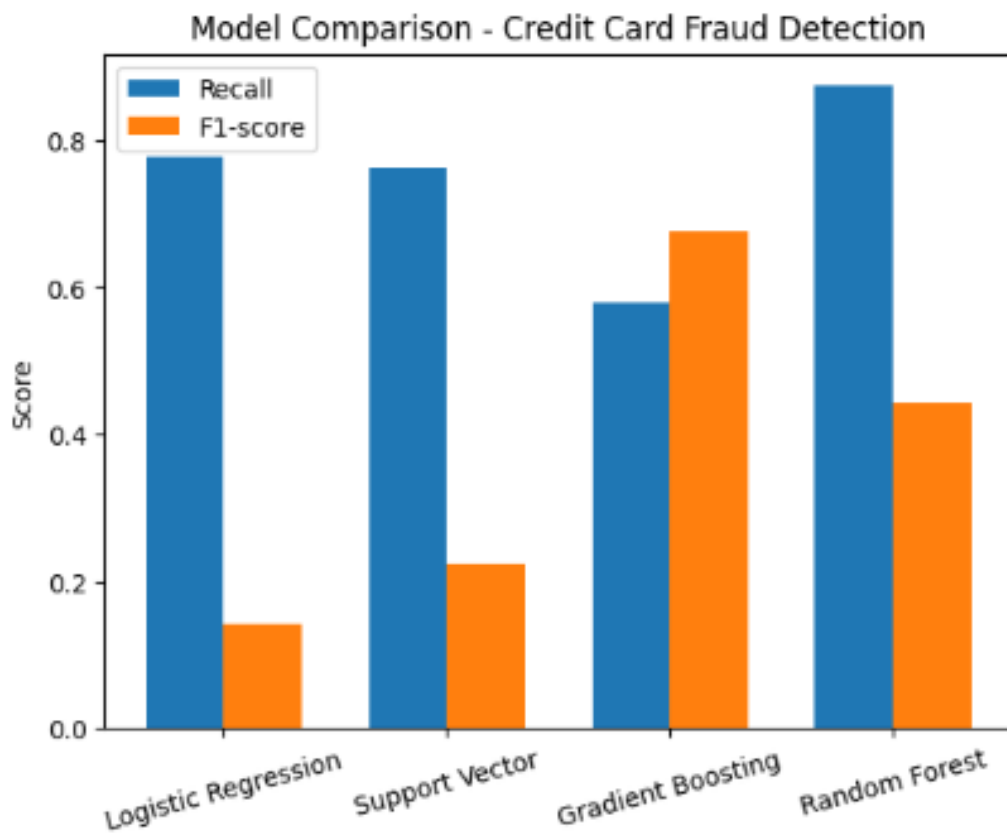
Models Implemented

- Logistic Regression
- Support Vector Machine
- Gradient Boosting
- Random Forest
- Combination of linear and ensemble models

```
lr = LogisticRegression(  
    C=0.5,  
    penalty='l2',  
    max_iter=300,  
    class_weight= 'balanced')  
  
svm = Pipeline([('scaler', StandardScaler()),  
    ('svm', LinearSVC(  
        C=1.0,  
        class_weight='balanced',  
        max_iter=5000,  
        random_state=37))])  
  
gb = GradientBoostingClassifier(  
    n_estimators=100,  
    learning_rate=0.05,  
    max_depth=3,  
    min_samples_split=30,  
    min_samples_leaf=15,  
    subsample=0.8)  
  
rf = RandomForestClassifier(  
    n_estimators=120,  
    max_depth=12,  
    min_samples_split=50,  
    min_samples_leaf=20,  
    max_features='sqrt',  
    class_weight='balanced',  
    n_jobs=-1, random_state=62)
```

Model Performance Comparison

- Logistic Regression recall ≈ 0.78 and F1 ≈ 0.14
- SVM recall ≈ 0.76 and F1 ≈ 0.22
- Gradient Boosting recall ≈ 0.58 and F1 ≈ 0.67
- Random Forest recall ≈ 0.88 and F1 ≈ 0.44
- Ensemble models outperform linear models



Recall Analysis

- Recall measures ability to detect fraud
- High recall reduces false negatives
- Random Forest detects approximately 88% fraud cases
- Logistic Regression detects approximately 78% fraud cases
- SVM detects approximately 76% fraud cases
- Gradient Boosting detects approximately 58% fraud cases
- Missing fraud is costly
- Recall prioritized over accuracy
- Business risk reduced with high recall
- Recall is a key evaluation metric

F1-Score Analysis

- F1-score balances precision and recall
- Gradient Boosting achieves highest F1-score of 67%
- Random Forest F1-score is 44%
- SVM F1-score is 22%
- Logistic Regression F1-score is 14%
- Low F1 indicates poor class balance
- F1-score is crucial for imbalanced data
- Precision impacts false alarms
- Balanced model preferred
- F1-score guides model selection

Final Model Selection

- **Gradient Boosting** is selected as the final fraud detection model
- Achieves the **best balance between recall and F1-score** among all models
- Effectively captures **non-linear transaction patterns**
- Performs reliably on **highly imbalanced fraud data**
- Reduces **false positives** while maintaining strong fraud detection
- Outperforms linear models and single decision trees
- Demonstrates **consistent performance across evaluation metrics**
- Suitable for **real-time and large-scale transaction systems**

Key Insights

- Overall fraud rate observed in the dataset is **5.78%**
- Fraud transactions have **significantly higher average amounts** than genuine ones
- **A small set of transaction categories** accounts for most fraud cases
- Fraud cases are **distributed across many merchants**, not concentrated in one
- **Transaction amount** is the most influential feature for fraud detection
- **Category information** adds important contextual risk signals
- The dataset shows **severe class imbalance**, impacting model evaluation
- **Ensemble machine learning models** deliver the best fraud detection performance

Conclusion

- Exploratory Data Analysis revealed **clear and meaningful fraud patterns**
- Fraud transactions are **rare but significantly high-value**, increasing financial risk
- Category-based behavior plays a **critical role** in identifying fraud
- Severe **class imbalance** makes accuracy an unreliable evaluation metric
- Recall and F1-score are **more suitable metrics** for fraud detection
- Machine learning models significantly improve detection capability
- Gradient Boosting provides the **best balance** between recall and precision
- The proposed approach is **practical for real-world deployment**
- Periodic retraining is essential to handle **evolving fraud patterns**

Thank You

- Thank you for your time and attention!
- I'm happy to answer any questions or discuss the analysis further.

Connect with me:

- **LinkedIn:**

<https://www.linkedin.com/in/amaan-uddin-18a476270/>

- **GitHub:**

<https://github.com/amaanuddin9318>

- **Dataset:**

[https://drive.google.com/file/d/19HtLaeXKYJunts6l2-SciyPaol5MFkF2/view?usp=drive link](https://drive.google.com/file/d/19HtLaeXKYJunts6l2-SciyPaol5MFkF2/view?usp=drive_link)

- **Project Code Pdf:**

https://drive.google.com/file/d/19uPWiy3ZEaxVpa2YZM5FNQ0Xy-vY1plZ/view?usp=drive_link